

Dokumentacja testowa

Do testowania aplikacji wykorzystaliśmy stworzony na backendzie Tester Endpointów.

Testowanie działania zostało przeprowadzone poprzez testowanie API, w tym celu przeprowadziliśmy testy funkcyjne, które pokrywają standardowe użycie aplikacji przez użytkowników.

Scenariusze testowe polegają na wprowadzeniu odpowiednich danych i ocenie odpowiedzi wysłanej przez serwer.

REST API Tester

Endpoint:

/cars

Metoda:

GET

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

name=Jan
position=Driver

Wyślij żądanie

Testowane endpointy:

Endpoint:

/cars

/employees

/cars

/profile

/stats/employee_of_month

/invoices

Przykładowe scenariusze testowe

1. Próba dodania pracownika.

REST API Tester

Endpoint:

/employees

Metoda:

POST

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

```
Name=Jan  
Surname=aaa  
Gender=M  
Date_of_birth=2000-01-01  
Employee_status_ID=2  
Car_dealer_ID=2  
Position_ID=2
```

Wyślij żądanie

Odpowiedź serwera:

```
Status: 201 CREATED  
  
{  
  "id": 101,  
  "message": "Employee created"  
}
```

Jak widać dodawany jest nowy pracownik. W odpowiedzi serwer wysłał ID nowo utworzonego pracownika.

2. Próba dodania pracownika o nieodpowiedniej wypłacie, czyli takiej nie mieszczącej się w określonych widełkach dla jego pozycji/stanowiska pracy.

REST API Tester

Endpoint:

/employees

Metoda:

POST

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

```
Name=Jan
Surname=aaa
Gender=M
Salary=200
Date_of_birth=1963-08-25
Phone_number=692137420
Employee_status_ID=1
Car_dealer_ID=1
Position_ID=1
```

Wyślij żądanie

Odpowiedź serwera:

```
Status: 400 BAD REQUEST

{
  "error": "Salary too low. "
}
```

Jak widać serwer wysłała informację o niespełnionym warunku.

3. Próba dodania pracownika o pozycji nieodpowiadającej żadnej pozycji zawartej w tabeli "Position"

REST API Tester

Endpoint:

/employees

Metoda:

POST

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

```
Name=Jan
Surname=aaa
Gender=M
Salary=16000
Date_of_birth=1963-08-25
Phone_number=692137420
Employee_status_ID=1
Car_dealer_ID=1
Position_ID=1000
```

Wyślij żądanie

Odpowiedź serwera:

Status: 400 BAD REQUEST

```
{
  "error": "Invalid Position_ID"
}
```

4. Próba dodania pracownika o nieprawidłowym formacie numeru telefonicznego.

REST API Tester

Endpoint:

/employees

Metoda:

POST

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

```
Name=Jan  
Surname=aaa  
Gender=M  
Salary=16000  
Date_of_birth=1963-08-25  
Phone_number=997  
Employee_status_ID=1  
Car_dealer_ID=1  
Position_ID=1
```

Wyślij żądanie

Odpowiedź serwera:

```
Status: 400 BAD REQUEST  
  
{  
  "error": "Invalid phone number format. "  
}
```

5. Próba fechtowania listy pracowników.

REST API Tester

Endpoint:

/employees

Metoda:

GET

ID zasobu (opcjonalne):

np. 1

Dane (klucz=wartość, rozdzielone enterem):

Wyślij żądanie

Odpowiedź serwera:

Status: 200 OK

```
[ ]
```

W odpowiedzi serwer wysyła pustą listę pracowników, oznacza to że nie ma żadnych pracowników. W przypadku, gdy są dodani pracujący pracownicy odpowiedź wygląda w następujący sposób.

Odpowiedź serwera:

Status: 200 OK

```
[
  {
    "Car_dealer_ID": 2,
    "Date_of_birth": "2000-01-01",
    "Employee_ID": 104,
    "Employee_status_ID": 1,
    "Gender": "M",
    "Login_credentials_ID": 2,
    "Name": "Jan",
    "Phone_number": null,
    "Position_name": "Manager",
    "Salary": 17968,
    "Status_name": "Active",
    "Surname": "aaa"
  }
]
```